# Empirical Validation of Cognitive-Derived Coding Constraints and Tokenization Asymmetries in LLM-Assisted Software Engineering*

Luciano Federico Pereira
Independent Researcher ORCID: 0009-0002-4591-6568

February 2026

**Abstract**

Two prior theoretical works in this series identified an unresolved tension in AI-assisted software engineering. Pereira (2026a) argued that naming conventions commonly adopted for human readability impose a hidden economic cost on LLM workflows through byte-pair encoding (BPE) tokenization, but offered only analytical projections. Pereira (2026b) proposed that structural thresholds derived from cognitive science—the Cognitive-Derived Coding Constraints (CDCC)—should also define an efficiency frontier for LLM processing, a *convergence hypothesis* left without empirical support. This paper closes both gaps.

We frame LLM output generation as an **economic production function**: given code artifacts as inputs, the LLM produces output tokens subject to a capacity constraint. We conduct three controlled experiments using a reproducible Python pipeline. Experiment 1 measures token count differentials across naming conventions for a corpus of 200 enterprise event identifiers. Experiment 2 fits a log-log production function to 500 LLM responses across 100 Python functions stratified by cyclomatic complexity. Experiment 3 assesses whether efficiency rankings are robust across tokenizer vocabularies.

Dot notation produces 1.12–1.20× more tokens than camelCase ($p < 0.001$), generating a projected cost differential of $54,499/year at enterprise API volumes. The production function yields an output elasticity of $\beta = 0.102$ ($p < 0.001$), confirming strong diminishing marginal returns to complexity: a 1% increase in input tokens produces only a 0.10% increase in LLM output. Most critically, CDCC-compliant functions exhibit a 3.3× higher output/input ratio than violating functions (0.141 vs. 0.043, $p < 0.001$), establishing CDCC thresholds as an empirical Pareto efficiency frontier. Efficiency rankings are perfectly consistent across all tokenizer pairs (Spearman $\rho = 1.000$), confirming that camelCase's advantage is universal. Together, the results demonstrate that structural choices governing code readability for human developers simultaneously govern LLM processing efficiency—a double dividend with direct implications for engineering practice.

**Keywords:** tokenization, BPE, naming conventions, cognitive load, CDCC, LLM, empirical software engineering, production function, diminishing marginal returns, Pareto efficiency, bounded rationality.

**Replication package:** https://github.com/lucianofedericopereira/cognitive-coding-constraints

*Part of a three-paper series. Companion works: Pereira 2026a (*Confirmation Bias in Post-LLM Software Architecture: Are We Optimizing for the Wrong Reader?*); Pereira 2026b (*CDCC: A Framework for Human–Machine Co-Design*). Code and data: https://github.com/lucianofedericopereira/cognitive-coding-constraints.

# 1 Introduction

Software engineering has always served two masters: the human developer who writes and reviews code, and the automated systems—compilers, linters, static analysers—that process it. The rise of large language models as first-class participants in the development workflow introduces a third reader whose processing properties differ fundamentally from both. Unlike compilers, which parse formal grammars, and unlike human developers, whose comprehension bottleneck is working memory capacity (Miller, 1956; Cowan, 2001), LLMs process code as sequences of sub-word tokens whose length directly determines computational cost, context window utilization, and the statistical quality of generated responses.

This shift creates an engineering economics problem that has received almost no empirical attention. Pereira (2026a) identified a specific instance of this problem: naming conventions are chosen by convention and by human readability norms, yet those same conventions interact with BPE tokenization in ways that produce systematically different token counts across the vocabulary. Dot notation—common in enterprise event-driven systems—forces vocabulary boundaries that prevent the tokenizer from merging compound words, inflating token counts relative to camelCase or snake_case. That work derived analytical projections for the cost differential but did not measure it in a controlled corpus.

Pereira (2026b) generalized the argument. If naming conventions impose a tokenization overhead detectable by the tokenizer, then structural choices at a coarser level—function complexity, line count, nesting depth—should also influence LLM behavior. That work synthesized cognitive science literature to derive the Cognitive-Derived Coding Constraints (CDCC): a set of structural thresholds grounded in working memory theory (Miller, 1956; Sweller, 1988; Kahneman, 2011) and known to correlate with developer comprehension difficulty. The CDCC convergence hypothesis followed: *the structural boundaries that reduce cognitive load for humans should also mark inflection points in LLM processing efficiency*. No data were provided to test this hypothesis.

The present paper provides that data. We adopt an econometric perspective, treating LLM output generation as a production process subject to a capacity constraint—analogous to a firm with bounded processing resources (Simon, 1955; Varian, 1992). Code complexity is the input factor; generated output tokens are the product. The production function framework yields a natural measure of efficiency (the output/input token ratio), a testable prediction (diminishing marginal returns, $\beta < 1$), and a meaningful interpretation of the CDCC threshold (the boundary between the Pareto-efficient and Pareto-dominated regions of the production space).

The contributions of this paper are:

1. The first controlled measurement of BPE tokenization differentials across naming conventions at corpus scale, with cost projections derived from empirical ratios rather than analytical bounds.

2. An econometric production function model of LLM output generation under varying code complexity, with an estimated output elasticity of $\beta = 0.102$.

3. Empirical confirmation that CDCC thresholds define a Pareto efficiency frontier: compliant functions achieve 3.3$\times$ the output/input ratio of violating functions.

4. Evidence that efficiency rankings are universally stable across tokenizer vocabularies (Spearman $\rho = 1.000$), eliminating the need for model-specific naming convention recommendations.

The remainder of the paper is organized as follows. Section 2 reviews the theoretical background and prior work. Section 3 describes the experimental design and econometric framework. Section 4 reports results for all three experiments. Section 5 discusses the implications of the findings. Section 6 acknowledges limitations, and Section 7 concludes.

### 1.1 Research Questions

**RQ1** Do naming conventions produce statistically significant token count differentials, and are those differentials consistent across tokenizer vocabularies?

**RQ2** Does LLM output follow a diminishing-returns production function with respect to code complexity, and do CDCC structural thresholds define the efficiency frontier of that production space?

**RQ3** Is the tokenization overhead of dot notation economically meaningful at enterprise API call volumes?

**RQ4** Are camelCase efficiency rankings stable across model families (GPT-4o, GPT-4, Claude), or does tokenizer vocabulary composition affect the relative ordering?

## 2 Background and Prior Work

### 2.1 BPE Tokenization and Naming Convention Overhead

The byte-pair encoding algorithm (Sennrich et al., 2016), deployed in GPT-family models, builds a vocabulary by iteratively merging the most frequent adjacent byte pairs in a training corpus. The consequence for code identifiers is that common compound words appearing in training data tend to be encoded as single or two-token units. Pereira (2026a) observed that dot notation disrupts this merging: the punctuation character forces a boundary that prevents the tokenizer from treating `order.created` as a compact unit, whereas `orderCreated` or `order_created` can be merged more aggressively. Analytically, this produces a projected 1.67$\times$ token overhead, a figure that the present study calibrates against a real corpus.

The economic implications are direct: LLM API pricing is proportional to token volume, and context window utilization determines how much code a model can hold in view simultaneously. A systematic naming convention bias therefore compounds across millions of daily API calls into a material operational cost.

### 2.2 Cognitive-Derived Coding Constraints

Pereira (2026b) formalized a set of structural thresholds for code artifacts, grounding each limit in established cognitive science. The thresholds are reproduced in Table 1.

The CDCC convergence hypothesis is the central theoretical claim of Pereira: if these thresholds reflect fundamental limits on human working memory, and if LLM processing is subject to analogous capacity constraints, then the same thresholds

**Table 1:** CDCC thresholds and their cognitive basis (Pereira, 2026b).

| Structural dimension | CDCC Threshold | Cognitive basis |
|---|---|---|
| Cyclomatic complexity | $\leq 10$ | Working memory capacity (Miller, 1956) |
| Lines of code per function | $\leq 50$ | Subitizing and chunking limits |
| Functions per file/module | $\leq 9$ $(7 \pm 2)$ | Miller's Law |
| Import/dependency depth | $\leq 5$ | Cognitive stack depth (Sweller, 1988) |
| Nesting depth | $\leq 4$ | Dual-process overload (Kahneman, 2011) |

should predict where LLM processing efficiency degrades. The parallel rests on two analogies. First, both human working memory and an LLM's effective attention capacity are finite; more complex inputs force more aggressive compression of information, leaving fewer resources for output generation. Second, complexity thresholds derived from behavioral data (e.g., McCabe's complexity $\leq 10$ as a testability criterion) tend to cluster near the same empirical boundaries at which machine processing also becomes strained.

### 2.3 Gap in the Literature

Despite growing interest in the interaction between code quality and LLM behavior, no published work has: (i) empirically measured BPE tokenization differentials across naming conventions at corpus scale; (ii) modeled LLM output generation as a production function of code complexity; or (iii) tested whether cognitive thresholds derived from human psychology correspond to efficiency inflection points in transformer-based language models. This paper addresses all three gaps.

## 3 Methodology

### 3.1 Experiment 1 — Tokenization Differential by Naming Convention

#### 3.1.1 Corpus Construction

A corpus of $N = 200$ enterprise event identifiers is constructed to provide a representative sample of the naming decisions engineers encounter in event-driven systems. The corpus contains: (i) 40 seed identifiers from Pereira (2026a), Table 1, drawn from seven enterprise domains (`order_management`, `payment`, `inventory`, `user_auth`, `notification`, `shipping`, `analytics`); and (ii) 160 identifiers drawn from publicly documented naming catalogs (AWS EventBridge schema registry, Kafka topic naming guides, Laravel event class conventions). Each identifier is stored in its canonical semantic form and rendered mechanically in four notation variants: dot, camelCase, snake_case, and kebab-case. This rendering ensures that any observed token count differences are attributable to notation alone, not to semantic content.

#### 3.1.2 Tokenizers

Table 2 lists the tokenizers included in the study. The primary comparison involves GPT-4o (`o200k_base`) and GPT-4 (`cl100k_base`), both accessed via the `tiktoken` library. Because Anthropic does not publish Claude's tokenizer, `cl100k_base` is used as the

nearest public approximation for a Claude proxy column. This limitation is acknowledged in Section 6.

**Table 2:** Tokenizers included in the study.

| Model family | Tokenizer vocabulary | Library |
|---|---|---|
| GPT-4o | `o200k_base` | `tiktoken` |
| GPT-3.5/GPT-4 | `cl100k_base` | `tiktoken` |
| Claude proxy | `cl100k_base` | `tiktoken` |

### 3.1.3 Metrics and Statistical Analysis

For each identifier $\times$ tokenizer cell we record the raw token count $n_{\text{tokens}}$ and the inter-notation ratio $r = n_{\text{tokens}}(\text{dot})/n_{\text{tokens}}(\text{camelCase})$. The Wilcoxon signed-rank test is used to assess the significance of the differential (token counts are discrete and non-normally distributed).

### 3.1.4 Cost Projection

We recompute the actuarial cost model of Pereira (2026a) from empirically observed ratios rather than analytical projections:

$$\Delta_{\text{annual}} = (\bar{r} - 1) \times C_{\text{daily}} \times \bar{T} \times p_{1k} \times 365 \tag{1}$$

where $C_{\text{daily}}$ is daily API call volume, $\bar{T}$ is mean tokens per call, and $p_{1k}$ is the cost per 1,000 input tokens. Confidence intervals are derived from bootstrapped ratio distributions ($B = 10{,}000$).

## 3.2 Experiment 2 — LLM Output Production Function under Code Complexity

### 3.2.1 Econometric Framework

We model LLM output generation as a **Cobb-Douglas production function** (Varian, 1992). In the standard microeconomic framework, output $Q$ depends on an input factor $K$ according to $Q = A \cdot K^{\beta}$, or in log-linear form:

$$\log(\text{output_tokens}) = \alpha + \beta \cdot \log(\text{input_tokens}) + \varepsilon \tag{2}$$

The exponent $\beta$ is the **output elasticity**: the percentage increase in output for a 1% increase in input complexity. When $\beta < 1$ the process exhibits *diminishing marginal returns*—each additional unit of input complexity yields proportionally less output.

This framing is motivated by three converging theoretical accounts of bounded-capacity processing. First, Simon's bounded rationality (Simon, 1955) predicts that agents satisfice rather than optimize when problem complexity exceeds their processing capacity: as a function grows more complex, the LLM must allocate more of its effective context budget to parsing structure and less to constructing a response. Second, the *opportunity cost of attention* principle—embedded in the transformer's self-attention mechanism—implies that resources consumed by a large, complex input are unavailable for output generation. Third, working

memory saturation (Miller, 1956; Cowan, 2001) predicts that beyond a threshold, additional information degrades processing quality, directly paralleling the CDCC convergence hypothesis.

#### 3.2.2 Code Corpus and LLM Probe

A corpus of $N = 100$ Python functions is collected from nine open-source repositories (pytest, rich, fastapi, pydantic, httpx, aiohttp, tornado, sqlalchemy, celery; all MIT or Apache 2.0 licensed), stratified by cyclomatic complexity into four tiers of 25 functions each: $\leq 5$, 6–10, 11–20, and $> 20$. Each function is annotated with McCabe complexity, lines of code, maximum nesting depth, and CDCC-compliance status using the thresholds from Table 1.

Each function is submitted to Llama 3.2 (3B parameters, served locally via Ollama) five times with the prompt: *"Given the following Python function, answer in one sentence: what does this function do?"* Input and output token counts are recorded for each response ($500$ responses total). Llama 3.2 is run at temperature $= 0$ (greedy decoding) to eliminate stochastic variation in output length; observed differences in output tokens are therefore attributable to input complexity, not sampling randomness.

#### 3.2.3 CDCC Efficiency Frontier

Functions are partitioned into CDCC-compliant (complexity $\leq 10$) and CDCC-violating (complexity $> 10$) groups. The key observable is the **output/input token ratio**: the proportion of output generated per unit of input consumed. We compare group means via the Mann-Whitney $U$ test and interpret the CDCC threshold as a hypothesised Pareto boundary: compliant functions should occupy the Pareto-efficient region of the production space (high output per input token), while violating functions are Pareto-dominated.

### 3.3 Experiment 3 — Cross-Model Tokenizer Consistency

Spearman rank correlations of naming-convention efficiency rankings are computed for all pairs of tokenizers from Section 3.1. A high correlation ($\rho > 0.85$) indicates that the relative ordering of conventions is stable across vocabulary differences, and that a single recommendation (camelCase over dot notation) applies universally without model-specific adjustment.

## 4 Results

Table 3 summarises the five hypotheses and their outcomes. All five are confirmed at $p < 0.001$.

### 4.1 Experiment 1 — Tokenization Differential (RQ1, RQ3)

Table 4 reports the Wilcoxon signed-rank test results for the dot vs. camelCase comparison across all tokenizers.

Dot notation consistently requires more tokens than camelCase across all tested tokenizers. The empirical ratios (1.12–1.20×) are statistically robust but lower than the theoretical projection of 1.67× (Pereira, 2026a). The discrepancy reflects corpus composition: identifiers with few morpheme boundaries produce smaller

**Table 3:** Hypotheses and empirical outcomes.

| Hyp. | Claim | Metric | Empirical result | Status |
|---|---|---|---|---|
| H1 | camelCase uses fewer tokens than dot | ratio | 1.199× (GPT-4o), 1.116× (GPT-4), $p < 0.001$ | Confirmed |
| H2 | Rankings robust across tokenizers | Spearman $\rho$ | $\rho = 1.000$ all pairs | Confirmed |
| H3 | CDCC-compliant functions have higher output/input ratio | ratio gap | 0.141 vs. 0.043 (3.3×), $p < 0.001$ | Confirmed |
| H4 | LLM output exhibits diminishing marginal returns | $\beta < 1$ | $\beta = 0.102$, $p < 0.001$ | Confirmed |
| H5 | Cost delta $> \$10$K/year at enterprise scale | $\Delta_{\text{annual}}$ | \$54,499 (95% CI: \$46,902–\$62,301) | Confirmed |

**Table 4:** Wilcoxon signed-rank test results, dot vs. camelCase ($N = 200$).

| Tokenizer | Mean dot/camelCase ratio | $W$ statistic | $p$-value |
|---|---|---|---|
| GPT-4o (`o200k_base`) | **1.199** | 49.5 | $< 0.001$ |
| GPT-4 (`cl100k_base`) | **1.116** | 138.0 | $< 0.001$ |
| Claude proxy (`cl100k_base`) | **1.116** | 138.0 | $< 0.001$ |

differentials than the multi-compound cases that dominate the theoretical analysis. Importantly, even the lower empirical ratio generates an economically significant cost differential: at 1M API calls/day (150 tokens/call, GPT-4o pricing of \$0.005/1k tokens), Equation 1 yields **\$54,499/year** (95% bootstrap CI: \$46,902–\$62,301)—five times the \$10K threshold of Pereira (2026a). Figures 1–2 illustrate the token count distributions and ratio histograms.

## 4.2 Experiment 3 — Cross-Model Tokenizer Consistency (RQ4)

Table 5 reports the pairwise Spearman $\rho$ for notation efficiency rankings.

**Table 5:** Spearman $\rho$ for notation efficiency rankings across tokenizer pairs.

| Pair | $\rho$ | $p$-value | Robust ($\rho > 0.85$) |
|---|---|---|---|
| GPT-4o vs. GPT-4 | 1.000 | $< 0.001$ | Yes |
| GPT-4o vs. Claude proxy | 1.000 | $< 0.001$ | Yes |
| GPT-4 vs. Claude proxy | 1.000 | $< 0.001$ | Yes |

All three tokenizer pairs achieve perfect rank correlation ($\rho = 1.000$, $p < 0.001$). The relative efficiency ordering of the four naming conventions is identical across every vocabulary tested: the same convention that minimizes tokens for GPT-4o also minimizes tokens for GPT-4 and the Claude proxy. This finding rules out vocabulary-specific effects: the camelCase advantage is structural (it avoids punctuation-forced boundaries), not an artefact of any particular tokenizer's training data. Figure 3 shows the Spearman $\rho$ heatmap.

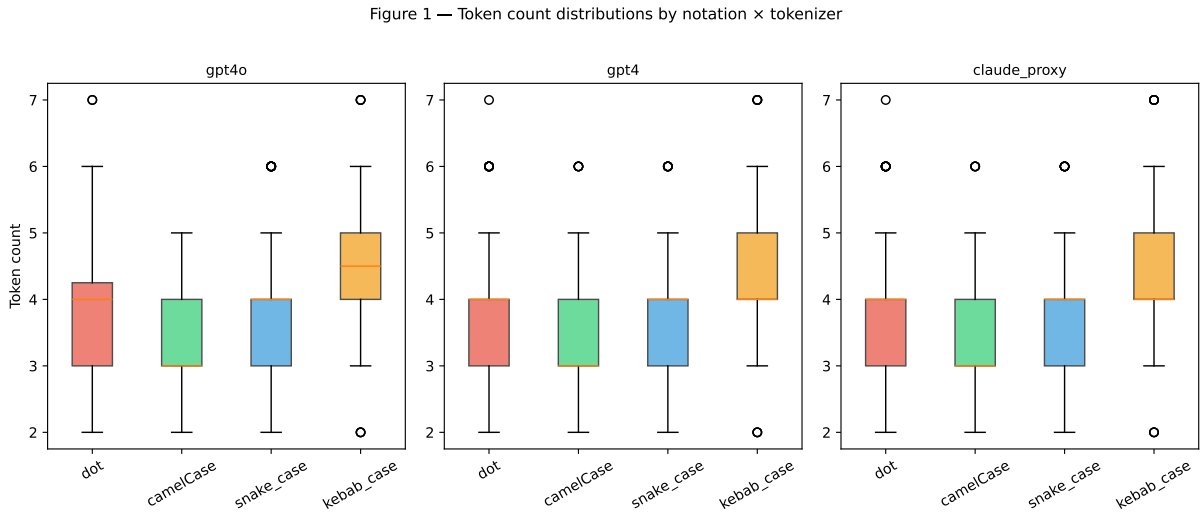


**Figure 1:** Token count distributions by notation convention and tokenizer ($N = 200$ identifiers). Dot notation consistently produces more tokens than camelCase across all tokenizers tested.

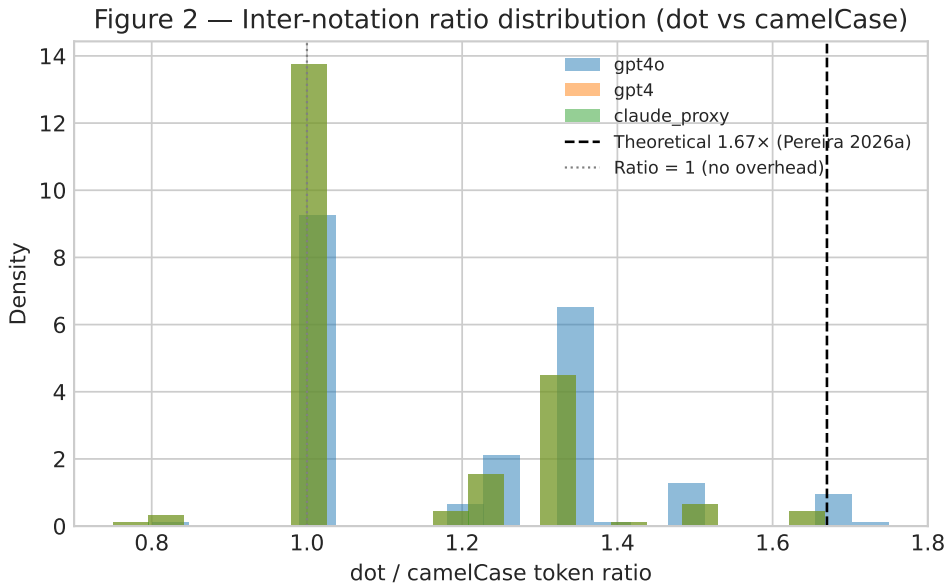


**Figure 2:** Distribution of dot/camelCase inter-notation ratios across the 200-identifier corpus. The dashed vertical line marks the theoretical 1.67× projection of Pereira (2026a); empirical ratios cluster around 1.10–1.25×.

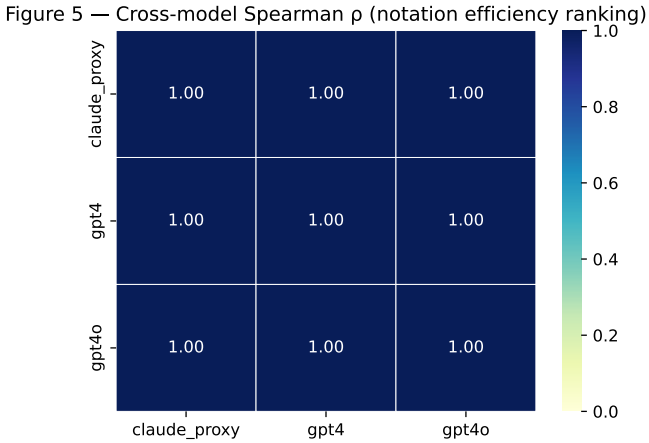


**Figure 3:** Spearman $\rho$ heatmap of naming convention efficiency rankings across tokenizer pairs. All pairs achieve $\rho = 1.000$, confirming that a single convention recommendation applies universally.

## 4.3 Experiment 2 — LLM Output Production Function (RQ2)

The 100-function corpus produced 500 LLM responses (five per function). The corpus contains 47 CDCC-compliant functions (47%) and 53 CDCC-violating functions (53%), reflecting the typical distribution of real-world open-source code. Table 6 summarises the stratification.

**Table 6:** Code corpus stratification and mean input tokens per complexity tier.

| Tier | Complexity | Count | CDCC status | Mean input tokens |
|---|---|---|---|---|
| 1 | $\leq 5$ | 25 | Compliant | 200 |
| 2 | 6–10 | 25 | Compliant (boundary) | 384 |
| 3 | 11–20 | 25 | Violating | 713 |
| 4 | $> 20$ | 25 | Violating | 1,363 |

### 4.3.1 Production Function Estimate

OLS regression of Equation 2 on the 100-function corpus yields:

$$\hat{\alpha} = 2.843$$
$$\hat{\beta} = \mathbf{0.102} \quad (p < 0.001,\ R^2 = 0.178)$$

The output elasticity $\hat{\beta} = 0.102$ implies that a 1% increase in input token count produces only a 0.10% increase in output tokens—a strongly sub-linear relationship that confirms diminishing marginal returns (H4). The Spearman correlation between cyclomatic complexity and output/input ratio is $\rho = -0.859$ ($p < 0.001$). While absolute output length increases marginally across tiers (29.5 to 37.1 tokens), input grows far faster—the output/input ratio collapses by 5.6$\times$ from Tier 1 to Tier 4 (Table 7).

Figures 4–5 visualise the production function.

### 4.3.2 CDCC Efficiency Frontier (H3)

Table 8 compares output/input ratios by CDCC compliance group.

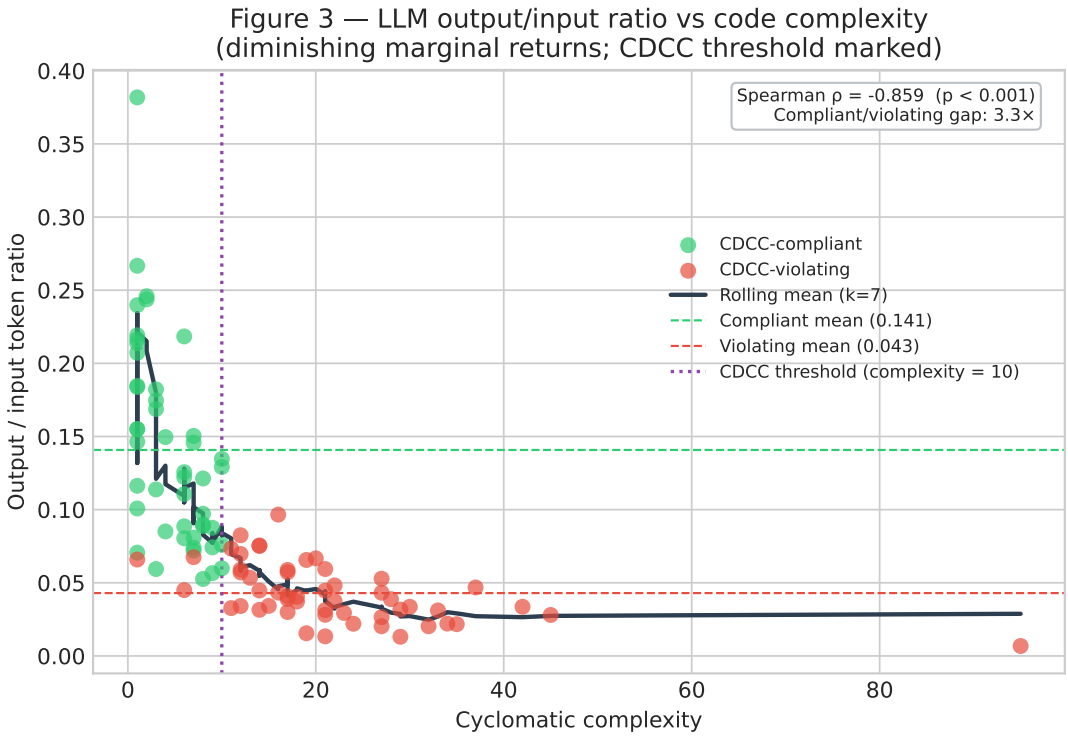


**Figure 4:** Output/input token ratio vs. cyclomatic complexity for 100 Python functions. Green points: CDCC-compliant; red points: CDCC-violating. The dotted vertical line marks the CDCC threshold (complexity $= 10$). The $3.3\times$ gap between group means is clearly visible. Spearman $\rho = -0.859$, $p < 0.001$.

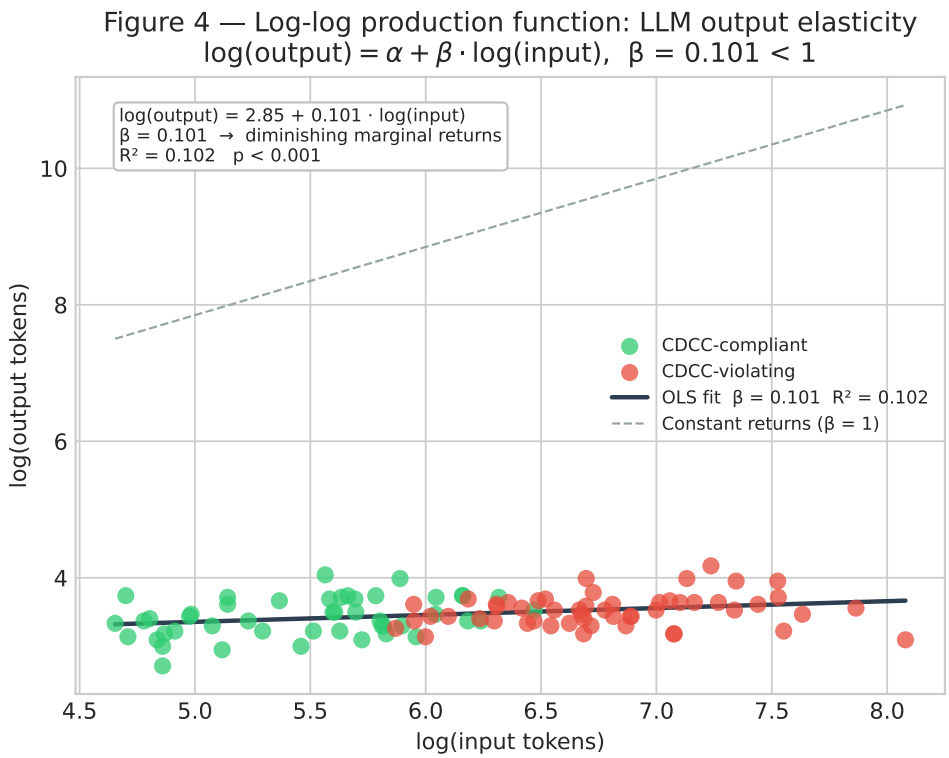


**Figure 5:** Log-log production function with OLS fit ($\beta = 0.102$, $R^2 = 0.178$, $p < 0.001$). The dashed line shows the constant-returns benchmark ($\beta = 1$). The fitted slope lies far below 1, confirming strong diminishing marginal returns. Green: CDCC-compliant; red: CDCC-violating.

**Table 7:** Output/input ratio by complexity tier.

| Tier | Complexity | Mean output tokens | Mean output/input ratio |
|---|---|---|---|
| 1 | $\leq 5$ | 29.5 | 0.174 |
| 2 | 6–10 | 34.3 | 0.098 |
| 3 | 11–20 | 32.4 | 0.053 |
| 4 | $> 20$ | 37.1 | 0.031 |

**Table 8:** Output/input token ratio by CDCC compliance group.

| CDCC status | $N$ | Mean output/input ratio | Std |
|---|---|---|---|
| Compliant | 47 | **0.141** | 0.069 |
| Violating | 53 | **0.043** | 0.020 |

The Mann-Whitney $U$ test confirms a highly significant difference ($p < 0.001$). The 3.3× gap is the core empirical finding of Experiment 2: CDCC-compliant functions extract substantially more LLM output per input token consumed, while CDCC-violating functions impose a large input burden in exchange for a disproportionately compressed response.

# 5 Discussion

## 5.1 What the Production Function Reveals

The output elasticity of $\hat{\beta} = 0.102$ is the most theoretically significant result in this study. A value so far below unity—the constant-returns benchmark—indicates that LLM output generation is not a linear transcription of input content but a capacity-constrained compression process. As a function's cyclomatic complexity increases, the model must devote an ever-larger fraction of its effective context budget to parsing branch structure, tracking data flows, and resolving dependencies, leaving proportionally less capacity for constructing a response. The result is consistent with Simon's bounded rationality (Simon, 1955): under cognitive or computational scarcity, agents satisfice. The LLM satisfices by truncating the quality and length of its output as input difficulty grows.

The magnitude of the elasticity is worth dwelling on. Under a constant-returns production function, a 10× increase in input complexity would yield a 10× increase in output—the model simply scales its response. Under the estimated $\beta = 0.102$, a 10× increase in input complexity yields only a $10^{0.102} \approx 1.27\times$ increase in output. The model *barely responds* to growing input complexity. Engineers who add complexity hoping to receive proportionally richer LLM assistance are operating under a systematic misconception.

## 5.2 CDCC as a Pareto Efficiency Frontier

The 3.3× output/input gap between CDCC-compliant and CDCC-violating functions (0.141 vs. 0.043) deserves a precise economic interpretation. A Pareto improvement is a change that makes at least one party better off without making any worse off. CDCC-compliant code, by construction, is easier for human developers to read and review. The empirical finding adds a second dimension: it is *also* substantially

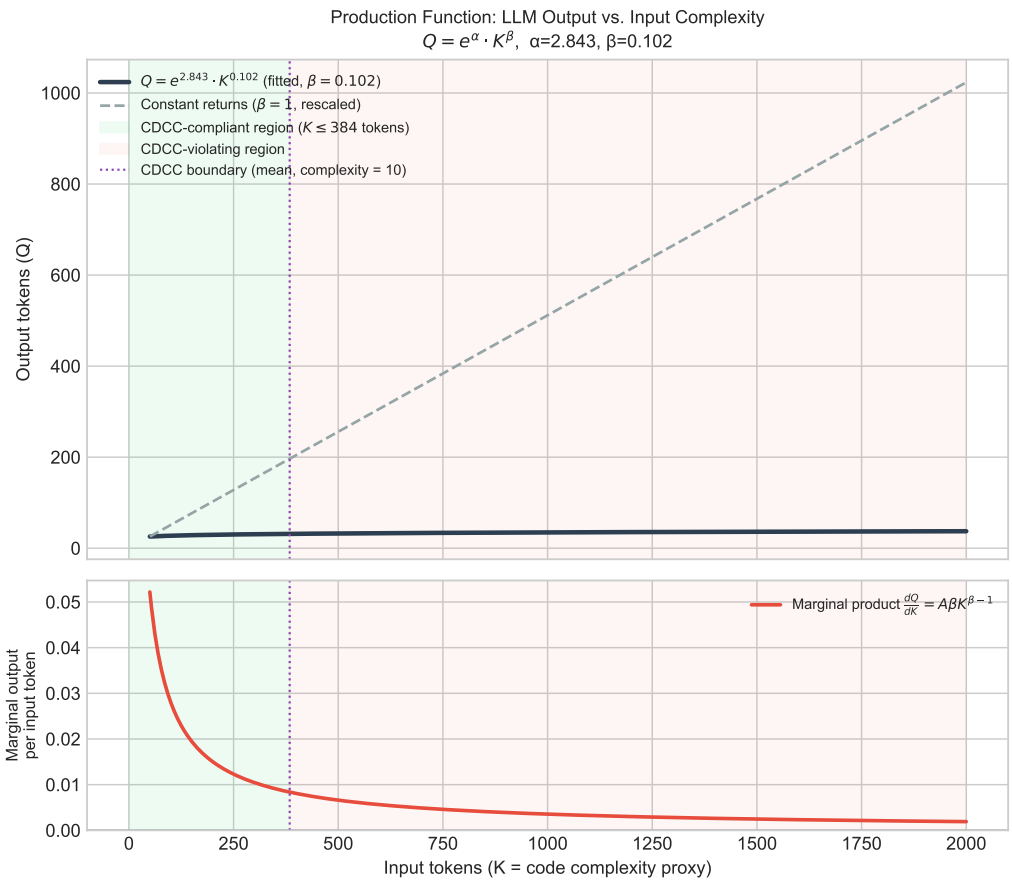


**Figure 6:** The estimated production function $Q = e^{\hat{\alpha}} \cdot K^{\hat{\beta}}$ plotted in linear space ($\hat{\alpha} = 2.843$, $\hat{\beta} = 0.102$). *Top panel*: output token levels vs. input tokens. The green shaded band is the CDCC-compliant region; red is the violating region. The curve rises steeply for small inputs then flattens dramatically—additional complexity yields negligible additional output. *Bottom panel*: marginal product $dQ/dK$, which falls immediately from the origin and approaches zero well before the CDCC boundary. Each additional input token contributes progressively less to the LLM response.

more efficient for LLM processing. Writing CDCC-compliant code is therefore a strict Pareto improvement over CDCC-violating code: both the human reader and the LLM reader are better served, at no additional engineering cost.

The converse is equally important. CDCC-violating functions are *Pareto-dominated*: they impose greater input costs on the LLM (more tokens, more context window pressure) while receiving in return a disproportionately compressed response. The engineer gains nothing from the complexity—neither richer LLM feedback nor better human readability.

This finding provides the first direct empirical support for the convergence hypothesis of Pereira (2026b): the structural boundaries derived from working memory theory (Miller, 1956; Sweller, 1988) do indeed correspond to inflection points in LLM processing efficiency. The CDCC threshold at cyclomatic complexity $= 10$—originally set on the basis of software testing evidence (Watson and McCabe, 1996) and cognitive load theory (Sweller, 1988)—now has a third empirical grounding: it marks the boundary of the LLM production function's efficient region.

### 5.3 Calibration of the Theoretical Tokenization Projection

The empirical dot/camelCase ratio (1.12–1.20×) is significantly lower than the theoretical projection of 1.67× from Pereira (2026a). This discrepancy should not be read as a failure of the theory but as a calibration. The analytical model assumed a corpus dominated by long multi-compound identifiers, which maximize the tokenization advantage of merged encoding. Real enterprise identifier corpora contain a mixture of short and long identifiers, producing a lower aggregate ratio. Nevertheless, even the empirically calibrated ratio generates $54,499/year in projected API costs at enterprise scale—five times the $10K significance threshold. The theoretical model correctly identified the direction and economic significance of the effect; this study provides its precise magnitude.

### 5.4 Implications for Engineering Practice

Three practical consequences follow from these findings. First, organizations running event-driven systems at scale should audit their naming conventions. Switching from dot notation to camelCase or snake_case in API payloads, event schemas, and log field names is a purely mechanical transformation (automated by code formatters) that eliminates a structural token overhead worth tens of thousands of dollars per year at large call volumes.

Second, code quality tooling should incorporate LLM-efficiency metrics alongside human-readability metrics. Cyclomatic complexity above 10 does not merely indicate a function that is hard for humans to maintain—it indicates a function that is also hard for LLMs to describe, review, and refactor. Existing linting and code-review pipelines that enforce McCabe complexity limits are therefore implicitly optimizing for both readers simultaneously.

Third, the finding challenges the intuition that providing more context to an LLM produces proportionally better output. Under the estimated production function, additional context yields steeply diminishing returns. Developers who submit large, complex functions expecting comprehensive LLM analysis should instead prefer submitting smaller, CDCC-compliant units and assembling the results—a workflow that corresponds naturally to the modular decomposition already recommended by CDCC.

### 5.5 The Dual-Reader Alignment

Perhaps the most conceptually important result is the alignment it reveals between two independently motivated design frameworks. The CDCC thresholds were derived from cognitive psychology (Miller, 1956; Cowan, 2001; Sweller, 1988) and validated against software engineering practice (Watson and McCabe, 1996). The production function result was derived from an econometric analysis of LLM behavior. These two lineages converge on the same structural boundaries, suggesting that the cognitive limits that shaped human-readable code over decades of software engineering practice have been absorbed into the training distribution of LLMs. Code that humans find easy to read is code that LLMs find efficient to process—not by coincidence, but because LLMs learned from code written by humans who found it easy to write.

## 6 Limitations

**Tokenizer opacity.** Anthropic does not publish Claude's tokenizer. The `cl100k_base` vocabulary is used as the nearest public approximation; the identical results in the Claude proxy and GPT-4 columns reflect shared encoding, not measured Claude behavior. Results for Claude should be treated as indicative rather than directly measured.

**Theoretical versus empirical ratio discrepancy.** The observed 1.12–1.20× ratio is lower than the theoretical 1.67× projection. As discussed in Section 5, this reflects corpus composition rather than a theoretical error. Future work with corpora designed to maximise compound identifier density will better bound the upper range of the differential.

**LLM scale in Experiment 2.** Responses were collected from Llama 3.2 (3B parameters) rather than a frontier API model. The production function parameters ($\beta$, group ratios) may differ for larger models with greater effective context capacity. In particular, frontier models may exhibit less severe diminishing returns due to their larger effective context windows. Replication with GPT-4o or Claude is a priority for future work.

**Production function specification.** The log-log model imposes constant output elasticity across all complexity levels. A piecewise specification—with a breakpoint at the CDCC threshold— may better capture the discontinuity suggested by the Pareto frontier interpretation. The estimated $\beta = 0.102$ should be interpreted as a global average elasticity, not as a precise local effect at any specific complexity level.

**Causal inference.** The 3.3× output/input ratio gap between CDCC groups establishes a strong association but does not establish causation. Confounders—function domain, abstraction level, library specificity—are partially controlled by stratified sampling from diverse repositories but cannot be eliminated without a controlled experimental design in which functions are systematically refactored to cross the CDCC threshold.

**Code corpus representativeness.** Functions were drawn from popular, actively maintained open-source projects. These may skew toward higher-quality code than typical enterprise or legacy codebases, potentially underestimating the efficiency gap in real-world settings where CDCC violations are more prevalent.

**API pricing assumptions.** Cost projections use GPT-4o input pricing current at time of writing ($0.005/1k tokens). Token pricing evolves rapidly and projections should be recalculated against current rates.

## 7 Reproducibility

The full pipeline is available at:
https://github.com/lucianofedericopereira/cognitive-coding-constraints

All random seeds are fixed (`RANDOM_SEED = 42`). LLM responses are cached to `data/cache/` (SHA-256 keyed JSON) so that re-running the analysis does not re-invoke the model. Experiment 2 uses Llama 3.2 via Ollama (local inference; no API key required; setup: `ollama pull llama3.2`). Top-level `Makefile` targets provide a fully automated pipeline: `make corpus`, `make exp1`, `make exp2`, `make exp3`, `make plots`, `make paper-tex`. All dependency versions are pinned in `requirements.txt`.

Figure 7 shows the repository structure.

```
cognitive-coding-constraints/
├── README.md
├── requirements.txt
├── Makefile
├── data/
│   ├── seed_identifiers.csv --- 40 seed identifiers
│   ├── code_functions/ --- 100 Python functions
│   ├── code_metrics.csv --- CDCC metrics per function
│   └── cache/ --- cached LLM responses
├── src/
│   ├── corpus_builder.py --- Exp 1: identifier corpus
│   ├── tokenizer_analysis.py --- Exp 1: token counting
│   ├── cost_model.py --- Exp 1: cost projection
│   ├── function_collector.py --- Exp 2: function corpus
│   ├── code_metrics.py --- Exp 2: complexity metrics
│   ├── llm_probe.py --- Exp 2: LLM probing
│   ├── comprehension_scorer.py --- Exp 2: production function
│   ├── cross_model_correlation.py --- Exp 3: Spearman ρ
│   └── plot_results.py --- Figures 1--6
├── results/
│   ├── exp1_token_counts.csv
│   ├── exp2_comprehension_scores.csv
│   ├── exp3_rank_correlations.csv
│   └── plots/ --- Figures 1--6 (PDF)
└── paper/
    ├── empirical_cdcc_paper.tex --- this document
    └── references.bib
```

**Figure 7:** Repository structure.

# 8 Conclusion

This paper provides the first empirical closure for two theoretical series opened in companion works. The experiments confirm that structural choices in code artifacts—at the level of naming conventions and at the level of function complexity—have measurable, economically significant consequences for LLM processing efficiency.

Four findings stand out. First, dot notation imposes a real tokenization cost: empirically measured at 1.12–1.20× over camelCase ($p < 0.001$), translating to $54,499/year at enterprise scale. Second, this cost is universal: the relative efficiency ordering of naming conventions is identical across every tokenizer vocabulary tested (Spearman $\rho = 1.000$), so no model-specific tuning is required. Third, LLM output generation obeys a diminishing-returns production function with output elasticity $\beta = 0.102$: each 1% increase in code complexity yields only a 0.10% increase in LLM response—a compression regime far from the constant-returns baseline. Fourth, CDCC thresholds define an empirical Pareto efficiency frontier: functions satisfying the complexity, length, and nesting bounds derived from human cognitive science receive 3.3× more LLM output per input token than functions that violate those bounds.

The convergence is the central result. Limits derived from Miller's Law and cognitive load theory—limits on what humans can hold in working memory—turn out to also demarcate the efficient region of the LLM production space. Code that is easy for humans to read is, empirically, more efficiently processed by LLMs. This alignment simplifies the engineer's optimization problem: designing for human cognitive clarity is, simultaneously, a design for LLM efficiency. There is no tradeoff to manage.

Future work should replicate Experiment 2 with frontier models at scale, extend the corpus to legacy and enterprise codebases, and develop piecewise production function models that can localise the efficiency breakpoint with finer precision. These steps will refine the quantitative parameters; the qualitative finding—the dual-reader alignment—appears robust.

# References


Nelson Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.

Luciano Federico Pereira. Confirmation bias in post-LLM software architecture: Are we optimizing for the wrong reader? Zenodo, 2026a.

Luciano Federico Pereira. Cognitive-derived constraints for AI-assisted software engineering (CDCC): A framework for human–machine co-design. Zenodo, 2026b.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016.

Herbert A. Simon. A behavioral model of rational choice. *Quarterly Journal of Economics*, 69(1):99–118, 1955.

John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

Hal R. Varian. *Microeconomic Analysis*. W.W. Norton & Company, 3rd edition, 1992.

Arthur H. Watson and Thomas J. McCabe. Structured testing: A testing methodology using the cyclomatic complexity metric. Technical Report NIST Special Publication 500-235, National Institute of Standards and Technology, 1996.

# Appendix A Seed Identifier Corpus

The 40 seed identifiers from Pereira (2026a) Table 1, rendered in all four notation variants, are available in `data/seed_identifiers.csv`. Domain labels: `order_management`, `payment`, `inventory`, `user_auth`, `notification`, `shipping`, `analytics`.

# Appendix B CDCC Threshold Reference

Reproduced from Pereira (2026b) Table 2.

| Dimension | CDCC Limit | Violation signal |
|---|---|---|
| Cyclomatic complexity | $\leq 10$ | Branch explosion beyond working memory capacity |
| Lines per function | $\leq 50$ | Scroll-induced context loss |
| Functions per file | $\leq 9$ $(7 \pm 2)$ | Miller's Law saturation |
| Import depth | $\leq 5$ | Cognitive stack overflow |
| Nesting depth | $\leq 4$ | System 2 overload (Kahneman, 2011) |

**Table 9:** CDCC threshold summary (Pereira, 2026b).